

— SYSTEM ARCHITECTURE SPEC

로컬 코딩 에이전트 오케스트레이션

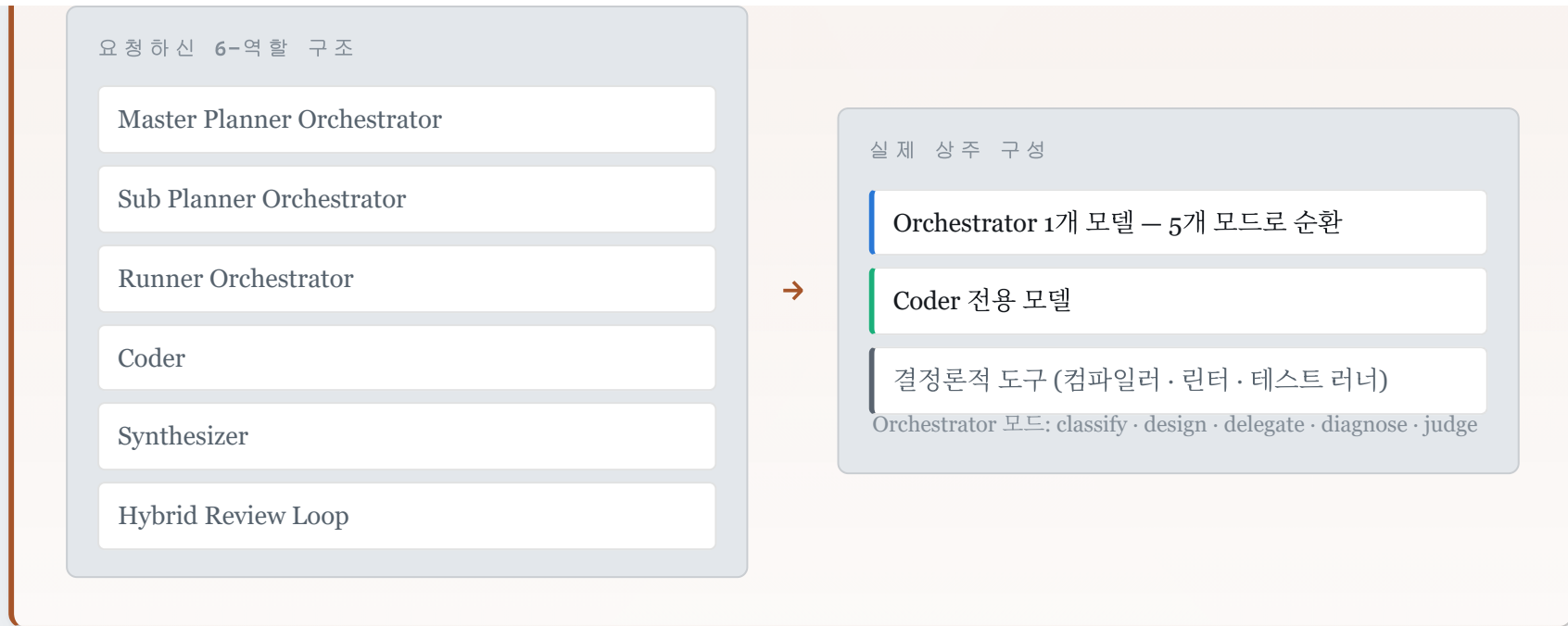
단일 RTX 4090(24GB) + 시스템 RAM 64GB 환경에서, 지연 속도와 성능을 동시에 확보하기 위한 추론 파이프라인 설계.

DOC	LOCAL-AGENT-ORCH
REV	2026-07
GPU	RTX 4090 · 24GB
RAM	64GB
OS	Windows 11

— 핵심 설계 결정

역할은 6개, 상주 모델은 2개면 충분하다

Master/Sub/Runner/Synthesizer/Hybrid Review — 이 다섯 단계는 전부 "판단" 작업이다. 코드를 직접 작성하는 Coder만 성격이 다르다. 다섯 개의 판단 역할마다 별도의 대형 모델을 오프로딩해 띄우면, 모델 스와핑과 디스크/RAM 스트리밍 지연이 누적되어 24GB VRAM 환경에서는 오히려 손해다. 대신 판단 역할을 모드가 다른 하나의 모델에 맡기고, 생성 전담 모델을 하나 더 두는 2-모델 구조를 권장한다.



PIPELINE

파이프라인 구조

아래 다이어그램은 논리적 6단계와, 그 뒤에서 실제로 움직이는 물리적 모델·도구의 매핑을 함께 보여준다. 실패 시 두 종류의 피드백 루프(즉시 재작성 / 재설계)가 존재한다.





■ Orchestrator (판단 · 5개 모드) ■ Coder (생성) ■ 결정론적 도구 (모델 아님) ■ 실패 피드백 루프

— ROLE MAPPING

단계별 역할과 실행 주체

단 계	수 행 작 업	실 행 주 체	비 고
1 · Master Planner	자연어 입력을 task 도메인으로 분류	Orchestrator · classify	출력은 짧은 분류 토큰 — 지연 영향 작음
2 · Sub Planner	도메인별 디렉터리 구조·코딩 계획 설계	Orchestrator · design	가장 긴 추론 — 262K 컨텍스트 활용
3 · Runner	계획을 파일 단위 프롬프트로 분해·위임	Orchestrator · delegate	function calling으로 Coder 호출 스키마 생성

단 계	수 행 작 업	실 행 주 체	비 고
4 · Coder	세부 프롬프트 기반 코드 작성	Coder 모델	GPU 큐 기반 순차 또는 소규모 배치 처리
5 · Synthesizer	의존성 설치 실행, 크로스 파일 정합성 진단	도구 + Orchestrator · diagnose	npm / pip / cargo 등은 도구가 직접 실행
6 · Hybrid Review	빌드 · 린트 · 테스트 실행 후 합격 판정	도구 + Orchestrator · judge	실패 원인에 따라 4 또는 3으로 회귀

MODEL SELECTION · 2026-07 기준

모델 추천

두 모델 모두 단독으로 24GB VRAM에 들어가는 것을 최우선 기준으로 선정했다 — 오프로딩 없이 최고 성능을 내는 조합.

Orchestrator (판단 · 5개 모드 겸용)

모 델	파 라 미 터	양 자 화	VRAM	비 고
Qwen3.6-35B-A3B 권장	35B (활성 3B) MoE	Q4	~20-22GB	24GB에 단독 상주, 오프로딩 불필요. 262K 네이티브 컨텍스트, thinking mode, function calling. 2026-04 출시, SWE-bench Verified 73.4%. Apache 2.0.
GPT-OSS-120B	120B (활성 5.1B) MoE	native MXFP4	~61GB	상한선용 대안. 24GB 단독 불가 — KTransformers류 하이브리드 오프로드 필요. 아래 "선택적 확장" 참고.

모 델	파 라 미 터	양 자 화	VRAM	비 고
GLM-4.5-Air	~106B MoE	4bit	확인 필요	활성 파라미터 수를 조사 과정에서 확정하지 못함 — 도입 전 공식 카드에서 직접 검증 권장.

Coder (생성 전담) – 두 가지 워크플로우 시나리오

"파일 하나당 최고 품질 워커 하나" vs "다수 워커를 동시에 돌려 Synthesizer가 병합"은 서로 다른 Coder를 요구한다. 관건은 모델 크기가 아니라 **토큰당 KV 캐시 크기**다 — 이게 작을수록 같은 24GB에서 더 많은 동시 슬롯이 들어간다. 아래는 각 모델의 공개 설정값(레이어 수·KV 헤드·어텐션 방식)을 직접 계산한 결과이며, 기존 30B 추천은 삭제하지 않고 시나리오별로 병기했다.

모 델	파 라 미 터	양 자 화	VRAM	어 텐 셴	8K 슬롯 (이 론 치)
Qwen3-Coder-30B-A3B 권장 · 단일 고품질 워커	30B (활성 3.3B) MoE	Q4_K_M	~22GB	GQA (KV헤드 4)	~4개
Qwen2.5-Coder-14B-Instruct	14B dense	Q4_K_M	~9-10GB	GQA (KV헤드 8)	~9개

모 델	파 라 미 터	양 자 화	VRAM	어 텐 셴	8K 슬롯 (이 론 치)
Qwen2.5-Coder-7B-Instruct	7B dense	Q4_K_M	~4-5GB	GQA (KV헤드 4)	~41개
DeepSeek-Coder-V2-Lite-16B 권장 · 고병렬 워크플로우	16B (활성 2.4B) MoE	Q4	~9GB	MLA (압축 512+64)	~57개

"8K 슬롯(이론치)"은 각 모델의 공개 설정값(레이어 수 · KV 헤드 또는 MLA 압축 차원)으로 토큰당 KV 캐시를 직접 계산해, 24GB에서 고정 오버헤드 1.5GB를 뺀 나머지를 8K 컨텍스트 슬롯 크기로 나눈 **메모리 기준 이론적 상한**이다. 실제로는 GPU 연산량이 메모리보다 먼저 병목이 되므로, RTX 4090 한 장에서 체감상 유효한 동시 슬롯 수는 이보다 낮다 — 정확한 수치는 실측이 필요하다.

시나리오 A · 순차 교체

Qwen3-Coder-30B-A3B

워커 1개 · 최고 개별 품질

파일을 한 번에 하나씩 처리

또는

시나리오 B · 고병렬 + MERGE

DeepSeek-Coder-V2-Lite-16B

워커 다수(체감 10-20개 추정) · 개별 품질은 낮음

Synthesizer의 병합 · 조율 비중이 커짐

모델 생태계는 몇 주 단위로 갱신된다. 위 표의 벤치마크 · 설정값 수치는 2026년 7월 기준 공개 자료를 인용 · 계산한 것으로, 도입 직전 공식 모델 카드에서 재확인할 것.

— RESOURCE ALLOCATION

메모리 할당 전략 — 순차 교체(Sequential Swap)

Orchestrator와 Coder를 동시에 올리지 않는다. 파이프라인이 원래 순차적이므로(계획 → 코딩 → 검증), 매 순간 **하나의 모델만 24GB 전체를 독점**하게 하면 오프로딩 자체가 필요 없어진다.

Phase A · 계획 단계 (1-3, 5, 6)

VRAM 24GB 중

Orchestrator 18GB

컨텍스트...

여유 3...

■ Orchestrator 가중치 ■ KV 캐시/런타임 ■ 여유

▶ 표로 보기

Phase B · 코딩 단계 (4)

VRAM 24GB 중

Coder 19GB

컨...

여유 3...

■ Coder 가중치 ■ KV 캐시/런타임 ■ 여유

▶ 표로 보기

전환(swap) 비용

모델 전환 시 디스크에서 재적재가 필요하지만, NVMe SSD + OS 페이지 캐시가 따뜻한 상태라면 수 초 내 완료된다. 아래처럼 RAM 대부분을 비워두는 것이 이 재적재 속도에 직접적으로 기여한다.

RAM · OS/상시 프로세스

~8GB

RAM · 페이지 캐시 여유

~56GB 모델 재적재 가속용

두 모델의 가중치 파일을 상시 RAM 페이지 캐시에 걸쳐두면(예열), 전환마다 SSD를 다시 읽지 않아도 되어 스왑 지연이 크게 줄어든다 — 64GB RAM에서 두 모델(~18GB + ~19GB ≈ 37GB)을 동시에 캐시하고도 여유가 남는다.

Coder 동시 슬롯 — "여유"의 재해석

Phase B의 컨텍스트 2GB + 여유 3GB(총 5GB)는 파일 한 개 몫의 여유가 아니라, `llama.cpp -parallel` 류 옵션으로 여러 파일을 동시에 생성하는 슬롯들이 나눠 쓰는 공유 풀로 다시 볼 수 있다. 슬롯당 컨텍스트를 8K 토큰 내외로 제한하면 추가 VRAM 없이 3-4개 슬롯을 동시에 띄워

처리량을 늘릴 수 있다. 정확한 슬롯 수는 모델의 레이어 · KV 헤드 구성에 따라 달라지므로, 보수적으로 시작해 VRAM이 찰 때까지 슬롯 수를 늘려가며 튜닝할 것.

시나리오 A vs B — Coder 메모리 배치 비교

어떤 Coder를 고르느냐에 따라 Phase B의 24GB가 완전히 다르게 나뉜다. 두 시나리오 모두 [순차 교체](#) 틀 안에서 성립한다 — 바뀌는 건 "Coder 차례일 때 몇 개를 동시에 돌리느냐"뿐이다.

시 나 리 오	CODER	가 중 치	슬롯 구 성 (8K 기 준)	소 요 VRAM	여 유
A · 단일 고품질	Qwen3-Coder-30B-A3B	19GB	1-4슬롯	~19-22GB	~2-5GB
B · 고병렬 + Merge	DeepSeek-Coder-V2-Lite-16B	9GB	16슬롯 예시(~4GB)	~13GB	~11GB (슬롯 추가 여력)

시나리오 B는 11GB 여유를 전부 슬롯에 쓰면 이론상 수십 개까지 늘어나지만(위 모델 표 참고), 실제로는 GPU 연산량이 먼저 병목이 된다. 16슬롯은 계산 편의를 위한 예시값이며, 실측하면서 슬롯 수를 조정할 것.

— OFFLOADING TECHNIQUE

AirLLM을 쓰지 않는 이유

애초에 이 설계에는 오프로딩이 필요하지 않다 — 두 모델 모두 24GB에 단독으로 들어가기 때문이다. 그럼에도 원 질문에서 명시하신 AirLLM에 대해 짚고 넘어간다.

기 법	설 계 목 적	이 파 이 프 라 인 과 의 궁 합
AirLLM	레이어 단위 디스크 스트리밍으로 초저용량 VRAM에서 대형 모델 1 회성 배치 추론	비권장 매 호출마다 스트리밍 비용을 다시 지불 — 순차 호출 이 많은 에이전트 루프에서 전체 GPU 상주 대비 10–50배 느려 짐
DeepSpeed-ZeRO-Inference / FlexGen	배치 처리량(throughput) 최적화	비권장 지연시간이 아닌 처리량 최적화 — 작은 요청이 반복 되는 대화형 루프엔 부적합
llama.cpp --n-gpu-layers	레이어를 GPU/RAM에 정적으로 분할	대안 조밀(dense) ~70B급 모델을 어쩔 수 없이 오프로드해 야 할 때의 실용적 기본값
llama.cpp --n-cpu-moe / KTransformers	MoE 라우팅 특성을 활용해 활성 전문가만 GPU에 유지	대안 GPT-OSS-120B급으로 확장하고 싶을 때만 사용 — llama.cpp 대비 1.25–1.93배 빠른 디코딩. Windows 네이티브 지원은 미확인, WSL2 권장

추측 디코딩(speculative decoding)도 성숙한 기술이지만(`llama.cpp --spec-draft-model`), CPU로 오프로드된 MoE 모델에는 역효과가 날 수 있다 — 드래프트 토큰이 라우팅하는 전문가가 매번 다르면 검증마다 새로 PCIe 페치가 발생해 오히려 느려진 사례(17→11 tok/s)가 보고됐다. 전량 VRAM 상주 모델에서만 사용할 것.

— TOOL STACK

도구 스택

구 성 요 소	선 택	비 고
오케스트레이션 프레임워크	LangGraph	DAG 네이티브, 상태 체크포인팅, 로컬 OpenAI 호환 엔드포인트 지향 가능. MetaGPT의 PM→Architect→Engineer→QA SOP 패턴을 프롬프트 설계 참고용으로 차용(유지보수 활성도는 미확인, 의존성으로는 비권장).
서빙 엔진	llama.cpp	두 모델 모두 MoE 포함 GGUF 지원 — 서버 모드로 OpenAI 호환 API 제공. Coder만 GPU 전용으로 최고 속도가 필요하면 ExLlamaV3 + TabbyAPI 대안.
실행 샌드박스	Docker Desktop (WSL2)	Windows 11에서 마찰이 가장 적은 격리 방식. "Docker Sandbox" 기능은 프로젝트 디렉터리만 마운트해 Coder에 과도한 권한을 주지 않는다.

말씀하신 두 저장소 검토

저 장 소	실 제 성 격	판 정
RyanCodrai/turbovec	Google TurboQuant 압축 기반 벡터 인덱스(ANN 검색) 라이브러리. LangChain/LlamaIndex 벡터스토어 대체용 — 오케스트레이션 도구가 아님.	선택적 코어 구조엔 불필요. 추후 코드베이스 RAG/컨텍스트 검색을 추가할 때만 고려.
google/agents-cli	Google Cloud(Vertex/Gemini Enterprise) ADK 기반 에이전트 빌드·배포 CLI. 로컬 개발도 Gemini API 키가 필요하고 배포는 GCP 프로젝트 전제.	제외 완전 로컬·비클라우드 요건과 정면으로 충돌.

— EFFICIENCY ADD-ONS

추가 효율화 레이어

아래 4개는 기존 2-모델 구조를 바꾸지 않고 얻을 수 있는 레이어다. 전부 CPU/RAM에서 돌거나 설정값 수준이라 VRAM 예산과 경합하지 않는다 — 별도 소형 모델을 추가로 상주시키는 방안은 아래에서 기각한 이유를 설명한다. Prefix 캐싱과 컨텍스트 매니페스트가 구현 비용 대비 효과가 가장 크다.

레이어	구현 방법	VRAM 영향
Prefix / Prompt 캐싱	Orchestrator의 5개 모드가 매년 공유하는 시스템 프롬프트 · 프로젝트 컨텍스트를 재계산 없이 재사용. llama.cpp 의 슬롯 기반 컨텍스트 재사용(--cache-reuse 류 옵션) 또는 vLLM의 automatic prefix caching.	0GB · 설정값
컨텍스트 매니페스트 (Librarian)	전체 파일 대신 시그니처 · 타입 · 의존관계만 추출해 유지. tree-sitter 기반 파싱 — Python/C++/Web 등 다국어 지원, LLM 미사용 순수 결정론적 도구.	0GB · CPU
Triage → 템플릿 분기	Runner의 delegate 출력에 "보일러플레이트 / 핵심 로직" 태그를 추가. 보일러플레이트는 템플릿 엔진이 직접 생성해 Coder 호출 자체를 생략하고, 핵심 로직만 Coder로 보낸다.	0GB · 도구
turbovec 시맨틱 캐시	소형 임베딩 모델(수백M 파라미터, CPU 추론 가능) + turbovec 압축 인덱스로 (요청 → 생성 결과) 캐시, 또는 기존 코드베이스 검색.	0GB · CPU/RAM

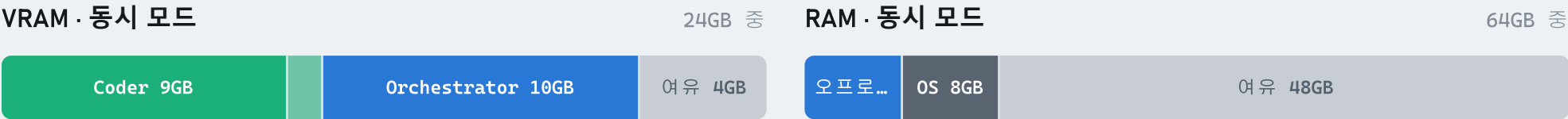
기각한 대안 — 전용 스캐폴드 모델

보일러플레이트 생성을 위해 별도의 소형 모델(1–3B급, 4비트에서 ~1GB)을 추가로 상주시키는 방안도 검토했다. 용량 자체는 들어가지만, Coder(~19GB)가 이미 VRAM을 거의 다 쓰는 상태에서 남는 헤드룸은 동시 배치 슬롯과 정확히 같은 자원을 두고 경합한다. 모델을 아예 쓰지 않는 템플릿 엔진 쪽이 그 헤드룸을 배치 슬롯에 온전히 넘겨줄 수 있어 더 낫다고 판단해 제외했다.

— OPTIONAL UPGRADE PATH

선택 사항 – 동시 파이프라이닝 모드

처리량이 더 중요하고 전환 지연조차 아깝다면, Coder를 상시 상주시키고 Orchestrator를 일부만 VRAM에 남긴 채 나머지를 RAM으로 오프로드하는 구성도 가능하다. 대신 더 작은 Coder를 써야 하고, 추측 디코딩은 위 이유로 제외한다.



이 모드는 Orchestrator가 계획을 세우는 동안 Coder가 이전 파일을 계속 작성할 수 있어 처리량이 늘지만, 오프로드된 전문가에 접근할 때마다 PCIe 지연이 붙는다. 단순성과 최고 속도를 원하면 위의 순차 교체 구조가 기본값이고, 이 모드는 처리량이 병목일 때만 켜는 것을 권장한다. GPT-OSS-120B급으로 Orchestrator 자체를 키우고 싶다면 이 자리에 KTransformers 하이브리드 오프로드를 대신 넣는다 (WSL2 필요, 지연 증가 감수).

모델 생태계와 로컬 추론 엔진은 몇 주 단위로 바뀐다. 이 문서는 2026년 7월 시점의 스냅샷이며, 특히 Orchestrator/Coder 모델 선택은 실제 도입 전 최신 벤치마크로 재확인할 것을 권장한다.

주요 출처

- 1. [Qwen3.6-35B-A3B · ModelScope](#)

2. [Qwen3.6-35B-A3B · OpenRouter 벤치마크](#)
3. [Qwen3-Coder · GitHub](#)
4. [Qwen2.5-Coder Technical Report](#)
5. [llama.cpp / KTransformers 오프로드 비교](#)
6. [llama.cpp 추측 디코딩 문서](#)
7. [FlexGen 논문](#)